

Метод нахождения проблемы: рисую график, на котором изображены "процессы" и относительное время их работы. Смотрю на тот процесс, который шире всех и ищу на `cppref` / `linux-gnu` ман особенности его работы

(1) jpeg

Проблема: количество аллокаций и соответственно количество вызовов деструктора для `vector<vector<vector>>>`. На картинках выделены alloc



2025-10-20T22:42:43+03:00

Running ./stock_jpeg

Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

L1 Data 32 KiB (x6)

L1 Instruction 32 KiB (x6)

L2 Unified 512 KiB (x6)

L3 Unified 4096 KiB (x1)

Load Average: 0.15, 0.11, 0.09

Benchmark	Time	CPU	Iterations
bm_scale_components	68.8 ms	68.8 ms	11

Решение: уменьшил число аллокаций путем:

- замены последнего слоя на array (так как точно знаем размер - это инвариант на всю программу)
- вместо поэлементного добавления на каждой итерации цикла, добавил resize сразу как только узнали это число (константы при входе в функцию) - эти изменения применил для обеих функций в программе: random_picture, scale_down



2025-10-21T12:40:31+03:00

Running ./jpeg

Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

L1 Data 32 KiB (x6)

L1 Instruction 32 KiB (x6)

L2 Unified 512 KiB (x6)

L3 Unified 4096 KiB (x1)

Load Average: 0.36, 0.19, 0.12

Benchmark	Time	CPU	Iterations
bm_scale_components	7.43 ms	7.19 ms	101

Прирост производительности: x12+

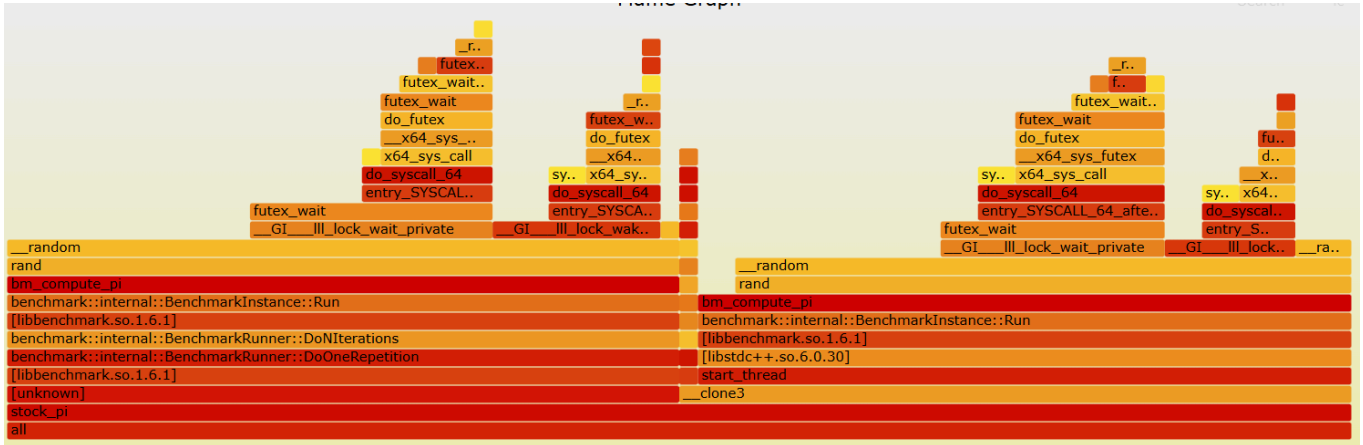
Забавные факты:

- в ходе исправления ошибки заметил, что выделять в 4 раза больше памяти одной аллокацией дольше чем выделять в 4 раза меньше последовательно (в этой

программе)

(2) pi

Проблема: в генераторе случайных чисел `std::rand`. Он в целом медленнее чем самая простая, наивная реализация, а с многопоточкой становится особенно медленным, так как для генерации, когда 2 потока будут просить случайные числа, в конечном итоге `std::rand` придется их синхронизировать, ставить в очередь.



2025-10-21T17:05:55+03:00

Running ./s-pi

Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

L1 Data 32 KiB (x6)

L1 Instruction 32 KiB (x6)

L2 Unified 512 KiB (x6)

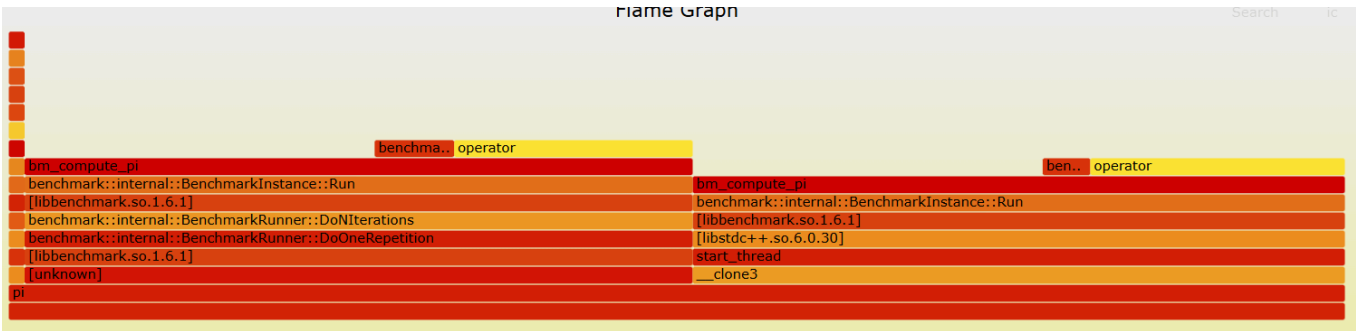
L3 Unified 4096 KiB (x1)

Load Average: 1.09, 0.43, 0.25

Benchmark	Time	CPU	Iterations
UserCounters...			
bm_compute_pi/threads:2	189 ns	358 ns	1963736 pi=3.1466

Решение: заменил генератор случайных чисел `std::rand` на ручную генерацию чисел (выбрал `xorshift`, так как он наиболее популярен в интернете). Это решит проблемы с синхронизацией потоков и очередями и в целом упростит генерацию с точки зрения

непосредственных вычислений.



2025-10-21T19:01:24+03:00

Running ./pi

Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

L1 Data 32 KiB (x6)

L1 Instruction 32 KiB (x6)

L2 Unified 512 KiB (x6)

L3 Unified 4096 KiB (x1)

Load Average: 0.79, 0.29, 1.94

Benchmark	Time	CPU	Iterations
UserCounters...			
bm_compute_pi/threads:2	1.96 ns	3.81 ns	196213362 pi=935.471u

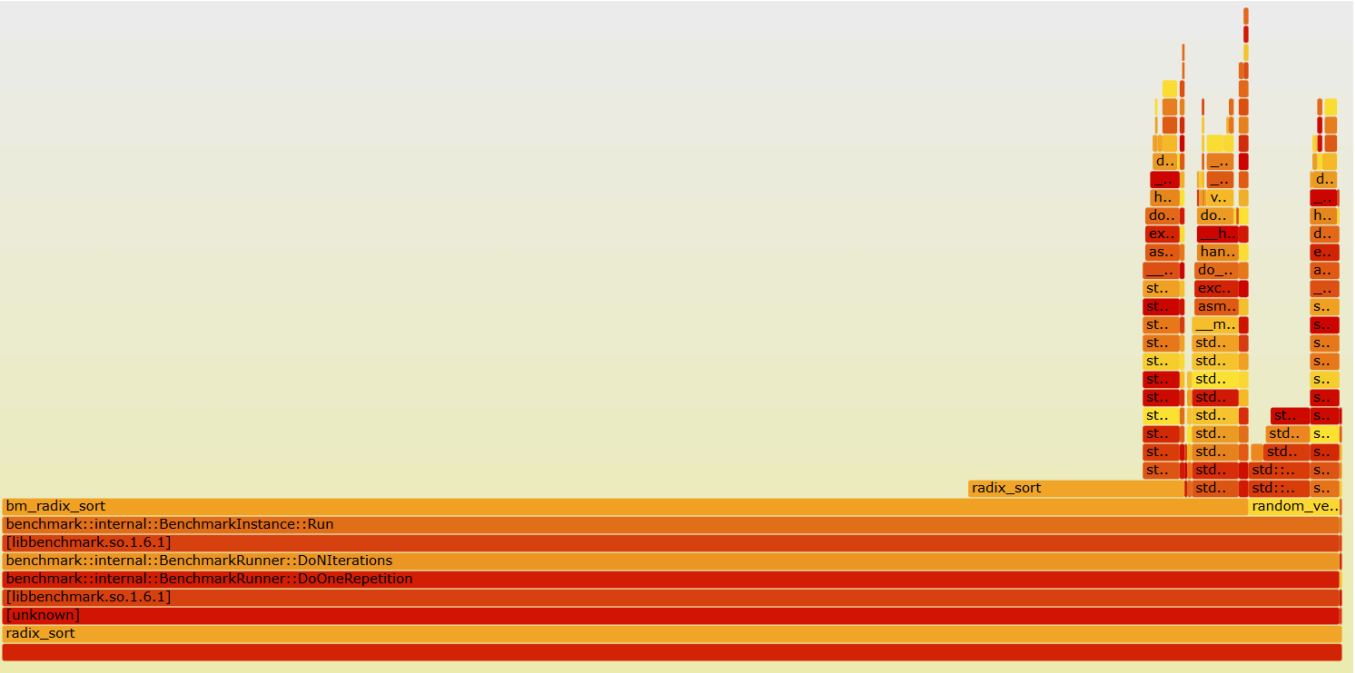
Прирост производительности: x100+-

Наблюдение: если заменить не на ручную реализацию, но на то, что не будет замедляться из-за многопоточки (например, std::mt19937 как используется в других тестах), то прирост в производительности составит x8

(3) radix-sort

Проблема: нерационально много созданий буферов в функции radix_sort (в каждой итерации). На картинках выделены alloc.

немного сэкономить (насколько я знаю это best-practice)



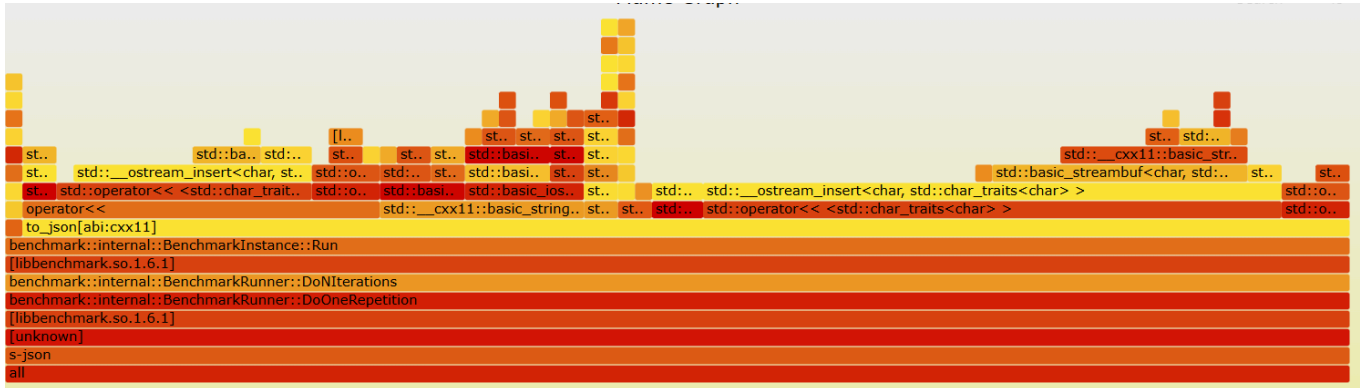
2025-10-21T19:54:30+03:00
Running ./radix_sort
Run on (12 X 2096.06 MHz CPU s)
CPU Caches:
 L1 Data 32 KiB (x6)
 L1 Instruction 32 KiB (x6)
 L2 Unified 512 KiB (x6)
 L3 Unified 4096 KiB (x1)
Load Average: 0.31, 0.10, 0.10

Benchmark	Time	CPU	Iterations
bm_radix_sort	5001 ms	4844 ms	1

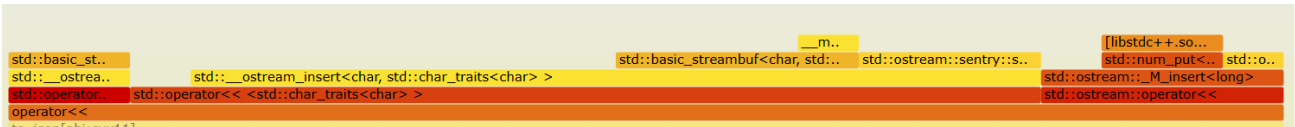
Прирост производительности: 50%+

(4) json

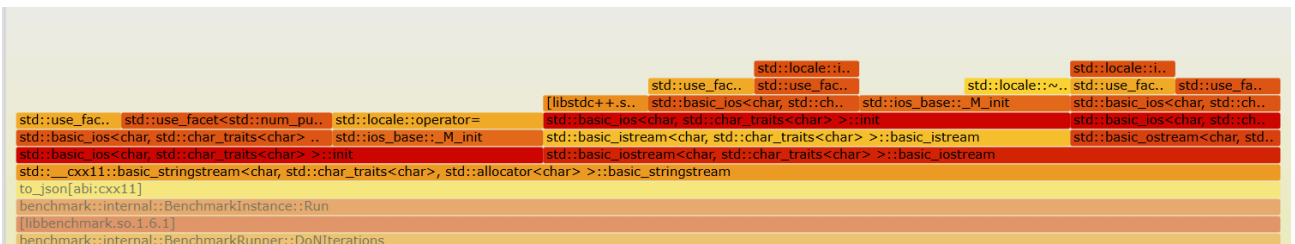
Проблема: слишком дорогой оператор <<. А именно:



- большое количество индирекций в процессе поиска нужной функции. Сначала нужно зайти в operator<<, затем в std::num_put::put(), затем походить по таблице виртуальных функций, чтобы найти реализацию метода для текущего класса, и только затем будет исполняться логика



- большое количество аллокаций: из-за последовательного добавления маленьких элементов буфер потока растет медленно и тратит на это лишние копирования, аллокации



2025-10-21T22:12:26+03:00

Running ./s-json

Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

L1 Data 32 KiB (x6)

L1 Instruction 32 KiB (x6)

L2 Unified 512 KiB (x6)

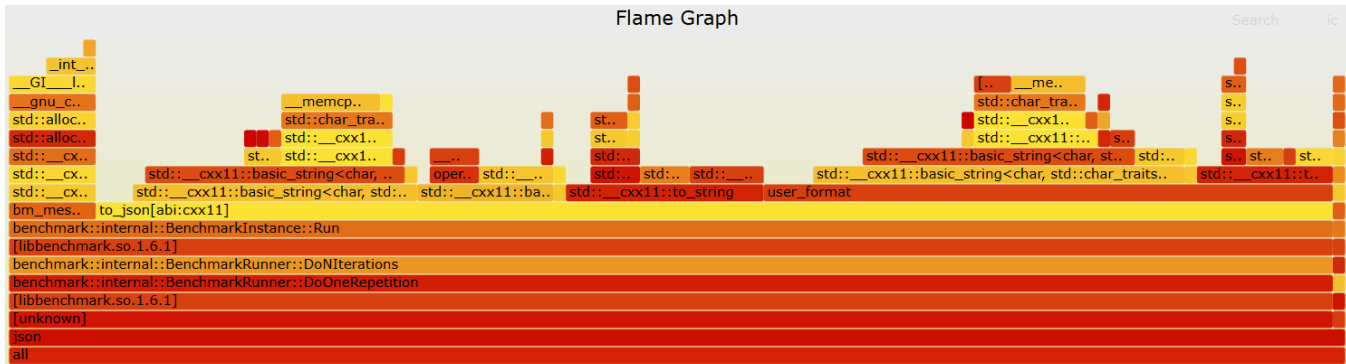
L3 Unified 4096 KiB (x1)

Load Average: 0.09, 0.16, 0.11

Benchmark	Time	CPU	Iterations
bm_message_to_json	1794 ns	1731 ns	426081

Решение: вместо потока std::stringstream, который использовал rdbuf для динамического расширения, использую std::string, для которой можно зарезервировать заранее (на входе

в функцию) память, тем самым избавляюсь от лишних аллокаций. Так же `append` в `std::string` дешевле по количеству индирекций чем оператор `<<`, так как не нужно ходить по виртуальным таблицам для поиска реализации.



2025-10-21T22:01:19+03:00

Running ./json

Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

L1 Data 32 KiB (x6)

L1 Instruction 32 KiB (x6)

L2 Unified 512 KiB (x6)

L3 Unified 4096 KiB (x1)

Load Average: 0.36, 0.23, 0.10

Benchmark	Time	CPU	Iterations
bm_message_to_json	344 ns	332 ns	1993214

Прирост производительности: x5+

(6) logger

Проблема: частый вызов write -> частые обращения к операционной системе, что дорого, т.к. таковы особенности работы с ОС (смена режима: из user в привилегированный,

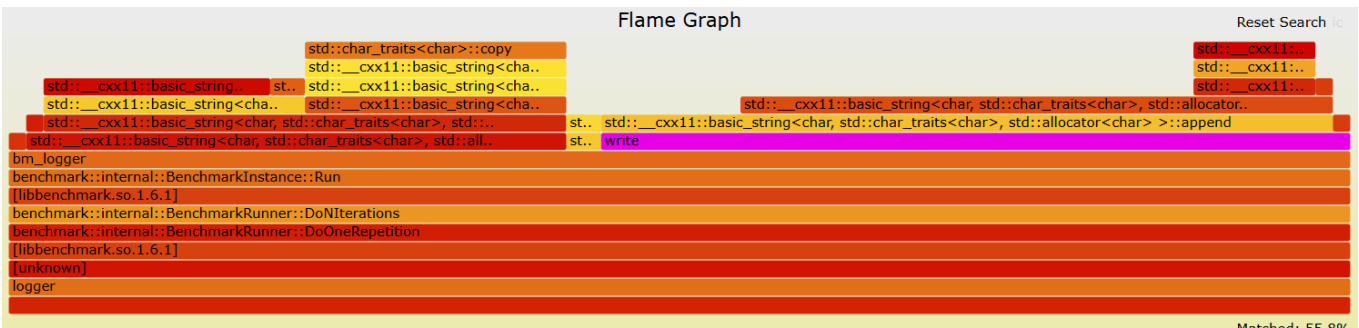
обратно). На графиках выделен "write"



2025-10-22T19:14:01+03:00
Running ./s-logger
Run on (12 X 2096.06 MHz CPU s)
CPU Caches:
 L1 Data 32 KiB (x6)
 L1 Instruction 32 KiB (x6)
 L2 Unified 512 KiB (x6)
 L3 Unified 4096 KiB (x1)
Load Average: 0.15, 0.13, 0.09

Benchmark	Time	CPU	Iterations
bm_logger/threads:1	3210 ns	3015 ns	234212

Решение: добавил буфферизацию, чтобы сократить число обращений к ОС для работы с файлом.



2025-10-22T19:13:28+03:00
Running ./logger
Run on (12 X 2096.06 MHz CPU s)

CPU Caches:

- L1 Data 32 KiB (x6)
- L1 Instruction 32 KiB (x6)
- L2 Unified 512 KiB (x6)
- L3 Unified 4096 KiB (x1)

Load Average: 0.18, 0.13, 0.10

Benchmark	Time	CPU	Iterations
bm_logger/threads:1	117 ns	109 ns	7841846

Прирост производительности: x27+